JavaFX con FXML y Scene Builder

Cómo crear la vista, el controlador y unir ambos

Programación Orientada a Objetos · DUOC UC
Proyecto: **2_javafx-fxml** · JavaFX 21 · JDK 21 · Maven 3.9

Índice

1. ¿Qué es FXML y por qué lo usamos?
2. Las piezas del proyecto y cómo se relacionan
3. Configurar el proyecto (pom.xml y module-info.java)
4. Instalar Scene Builder y conectarlo con IntelliJ
5. Paso a paso 1: la demo mínima (vista + controlador)
6. Cómo se unen la vista y el controlador (reglas de oro)
7. Paso a paso 2: el Login
8. Elementos y métodos importantes
9. Ejecutar por línea de comandos y con IntelliJ
10. Errores frecuentes
11. Resumen y checklist

1. ¿Qué es FXML y por qué lo usamos?

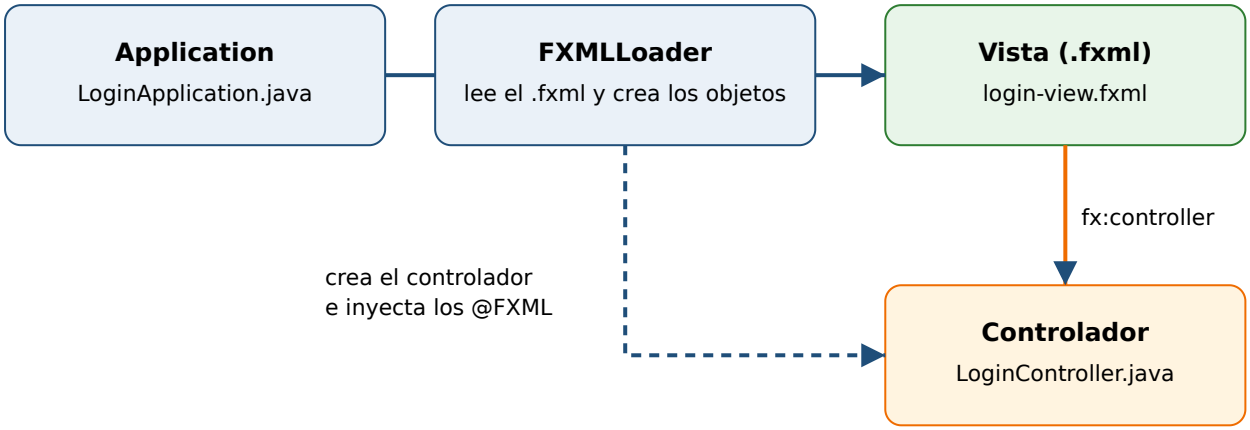
En el proyecto 1 construimos la ventana **escribiendo código Java** (`new Label(...)`, `new Button(...)`). Funciona, pero mezcla en una sola clase *cómo se ve* la ventana con *qué hace*.

FXML es un archivo XML donde se describe **solamente la apariencia** de la ventana: qué componentes hay y cómo están ordenados.

Forma	Vista (apariencia)	Lógica (qué hace)
Proyecto 1 (solo código)	Todo mezclado en `HelloApplication.java`	
Proyecto 2 (FXML)	`.fxml` (XML, editable con Scene Builder)	`Controller.java` (clase Java)

> **¿Para qué sirve?** Aplica el principio de **separación de responsabilidades** (la idea detrás del patrón MVC): un archivo se encarga de la vista y una clase del comportamiento. Así el diseño se puede cambiar sin tocar la lógica, y se puede diseñar **arrastrando y soltando** con Scene Builder en vez de escribir código.

2. Las piezas del proyecto y cómo se relacionan



Pieza	Archivo	Responsabilidad
Aplicación	`LoginApplication.java`	Arranca JavaFX y le dice qué vista cargar en la ventana.
Vista	`login-view.fxml`	Describe los componentes (diseñada con Scene Builder).
Controlador	`LoginController.java`	Contiene los métodos que responden a clics y manejan los datos.
FXMLLoader	(clase de JavaFX)	Lee el `.fxml`, crea los objetos y los conecta con el controlador.

Estructura del proyecto

```
2_javafx-fxml/
├── pom.xml
└── src/main/
    ├── java/
    │   ├── module-info.java
    │   └── com/duoc/javafxfxml/
    │       ├── HolaApplication.java        ← demo mínima
    │       ├── HolaController.java
    │       ├── LoginApplication.java       ← login
    │       ├── LoginController.java
    │       └── BienvenidaController.java
    └── resources/com/duoc/javafxfxml/      ← MISMO paquete que el código
        ├── hola-view.fxml
        ├── login-view.fxml
        └── bienvenida-view.fxml
```

Regla importante: los `.fxml` van en `src/main/resources` y en una carpeta con **el mismo nombre del paquete** (`com/duoc/javafxfxml`). Así `getClass().getResource("login-view.fxml")` los encuentra.

3. Configurar el proyecto

Es el mismo proyecto del documento 1, con **dos cambios** para poder usar FXML.

Cambio 1 · pom.xml: agregar la dependencia javafx-fxml

pom.xml (dependencias)

```
    <dependencies>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-controls</artifactId>
            <version>${javafx.version}</version>
        </dependency>
        <dependency>
            <groupId>org.openjfx</groupId>
            <artifactId>javafx-fxml</artifactId>
            <version>${javafx.version}</version>
        </dependency>
    </dependencies>
```

Antes solo estaba `javafx-controls`. `javafx-fxml` contiene `FXMLLoader` y las anotaciones como `@FXML`.

Además, cambiamos la clase principal por defecto a la del login, y se puede cambiar al ejecutar:

pom.xml (propiedades)

```
    <properties>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
        <maven.compiler.release>21</maven.compiler.release>
        <javafx.version>21.0.5</javafx.version>
        <main.class>com.duoc.javafxfxml.LoginApplication</main.class>
        <main.module>com.duoc.javafxfxml</main.module>
    </properties>
```

Cambio 2 · module-info.java

src/main/java/module-info.java

```
module com.duoc.javafxfxml {
    requires javafx.controls;
    requires javafx.fxml;

    // FXMLLoader usa reflexión para crear los controladores e inyectar los @FXML
    opens com.duoc.javafxfxml to javafx.fxml;
    exports com.duoc.javafxfxml;
}
```

Línea	¿Por qué?
`requires javafx.fxml;`	Necesitamos el módulo FXML para usar `FXMLLoader`.

`opens com.duoc.javafxfxml to javafx.fxml;`	FXMLLoader usa **reflexión** para crear el controlador y asignar los campos privados con `@FXML`. Hay que “abrir” el paquete para que pueda hacerlo.

> Si olvidas el `opens`, la app falla con `Module ... does not open ... to module javafx.fxml`.

4. Instalar Scene Builder y conectarlo con IntelliJ

Scene Builder es una herramienta gratuita (de Gluon) para diseñar archivos FXML arrastrando componentes. No es obligatoria (el FXML es texto y se puede escribir a mano), pero facilita mucho el diseño.

4.1 Descargar e instalar

Descarga desde `gluonhq.com/products/scene-builder` la versión **21** (la misma de JavaFX del proyecto).

Linux

- **Debian/Ubuntu:** descarga el `.deb` e instálalo:

  ```
  sudo dpkg -i SceneBuilder-21.x.x.deb
  ```

- **Sin permisos de administrador:** descarga el `.jar` y ejecútalo con:

  ```
  java -jar SceneBuilder-21.x.x-all.jar
  ```

- El programa queda en `/opt/scenebuilder/` (ejecutable `/opt/scenebuilder/bin/SceneBuilder`).

Windows

1. Descarga el instalador `.msi` y ejecútalo (siguiente → siguiente → instalar).
2. Queda en `C:\Users\TU_USUARIO\AppData\Local\SceneBuilder\SceneBuilder.exe`.

4.2 Conectarlo con IntelliJ

1. *File → Settings…* (en Windows/Linux) → *Languages & Frameworks → JavaFX*.
2. En **Path to SceneBuilder** escribe o busca la ruta del ejecutable (ver 4.1).
3. *Apply → OK*.
4. Para abrir un archivo: clic derecho sobre el `.fxml` → **Open in SceneBuilder**. También, al abrir el `.fxml` en IntelliJ, aparece abajo la pestaña **Scene Builder**.

> **Importante:** guarda en Scene Builder con `Ctrl + S`. IntelliJ recarga el archivo. Scene Builder **reordena e inserta código** en el FXML: es normal.

4.3 Conocer Scene Builder

Zona	Para qué sirve

Library (izquierda arriba)	Catálogo de componentes (Button, Label, VBox…). Se arrastran al centro.
Document → Hierarchy (izquierda abajo)	Árbol de componentes: quién contiene a quién.
Centro	Vista previa: el diseño de la ventana.
Inspector → Properties (derecha)	Propiedades: texto, fuente, ancho, estilos…
Inspector → Layout	Tamaño, márgenes, alineación, *padding*.
Inspector → Code	**fx:id** y eventos (**On Action**). Aquí se conecta con el controlador.
Controller (izquierda abajo)	Se escribe la clase controladora (`fx:controller`).

5. Paso a paso 1: la demo mínima

Objetivo: una ventana con un texto y un botón. Al presionar el botón, el texto cambia y cuenta los clics.

Paso 1 · Crear la vista en Scene Builder

1. En IntelliJ: clic derecho sobre la carpeta `src/main/resources/com/duoc/javafxfxml` → *New* → *File* → `hola-view.fxml`. (O desde Scene Builder: *File* → *New* y luego *Save* en esa carpeta.)
2. Abre el archivo en Scene Builder.
3. Arrastra un **VBox** desde *Library* → *Containers* hasta la ventana: será el contenedor raíz.
4. Arrastra un **Label** y un **Button** dentro del VBox (Library → Controls).
5. Selecciona el VBox. En *Inspector* → *Properties*: **Alignment = CENTER**, **Spacing = 20**. En *Layout*: **Padding = 20**.
6. Selecciona el Label: en *Properties* → *Text* escribe `Presiona el botón`. En *Code* → *fx:id* escribe **`mensajeLabel`**.
7. Selecciona el Button: en *Properties* → *Text* escribe `Saludar`. En *Code* → *On Action* escribe **`onSaludarClick`**.
8. Selecciona el VBox. En *Controller* (panel izquierdo abajo) → **Controller class**: `com.duoc.javafxfxml.HolaController`.
9. Guarda con `Ctrl + S`.

Paso 2 · Revisar el FXML que generó Scene Builder

hola-view.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.VBox?>

<VBox alignment="CENTER" spacing="20.0" xmlns="http://javafx.com/javafx/21"
xmlns:fx="http://javafx.com/fxml/1" fx:controller="com.duoc.javafxfxml.HolaController">
   <padding>
      <Insets bottom="20.0" left="20.0" right="20.0" top="20.0" />
   </padding>
   <children>
      <Label fx:id="mensajeLabel" text="Presiona el botón" />
      <Button onAction="#onSaludarClick" text="Saludar" />
   </children>
</VBox>
```

Parte del XML	Significado

`<?import ...?>`	Equivalente a los `import` de Java. Scene Builder los agrega solo.
`<VBox ...>`	El contenedor raíz. Es la clase `VBox`; sus propiedades se escriben como atributos.
`fx:controller="..."`	**Dice qué clase Java controla esta vista** (nombre completo con el paquete).
`<children>`	Los componentes hijos (equivale a `getChildren().add(...)`).
`fx:id="mensajeLabel"`	Nombre con el que el controlador puede usar este componente.
`onAction="#onSaludarClick"`	Método del controlador que se ejecuta al hacer clic. El `#` es obligatorio.

Paso 3 · Crear el controlador

HolaController.java

```
package com.duoc.javafxfxml;

import javafx.fxml.FXML;
import javafx.scene.control.Label;

public class HolaController {

    @FXML
    private Label mensajeLabel;

    private int clics = 0;

    @FXML
    private void onSaludarClick() {
        clics++;
        mensajeLabel.setText("¡Hola desde el controlador! Clics: " + clics);
    }
}
```

- El campo `mensajeLabel` se llama **igual** que el `fx:id`. JavaFX lo asigna automáticamente (*inyección*).
- El método `onSaludarClick` se llama **igual** que el valor de `onAction` (sin el #).
- `@FXML` permite que JavaFX acceda a campos y métodos **privados**. Así se mantiene el encapsulamiento.

Paso 4 · Cargar la vista desde la aplicación

HolaApplication.java

```
package com.duoc.javafxfxml;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;
```

```
import java.io.IOException;

/** Demo mínima: carga hola-view.fxml (la vista) que usa HolaController (el controlador).
*/
public class HolaApplication extends Application {

    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader loader = new FXMLLoader(getClass().getResource("hola-view.fxml"));
        Parent raiz = loader.load();

        stage.setTitle("Hola FXML");
        stage.setScene(new Scene(raiz, 320, 240));
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Línea	¿Qué hace?
`new FXMLLoader(getClass().getResource("hola-view.fxml"))`	Prepara el cargador indicando dónde está el archivo (relativo al paquete de la clase).
`loader.load()`	Lee el FXML, crea todos los objetos, crea el controlador y conecta todo. Devuelve el nodo raíz (`Parent`).
`new Scene(raiz, 320, 240)`	Mete la raíz dentro de una escena. Igual que en el proyecto 1.

Paso 5 · Ejecutar

```
mvn javafx:run -Dmain.class=com.duoc.javafxfxml.HolaApplication
```

Aparece la ventana. Cada clic en *Saludar* actualiza el texto.

6. Cómo se unen la vista y el controlador

La unión se hace por **nombres**. Si un nombre no coincide, falla. Estas son las tres reglas:

En la vista (.fxml)	En el controlador (.java)	Qué conecta
`fx:controller="com.duoc.javafxfxml.HolaController"`	La clase `HolaController`	Qué controlador usa la vista
`fx:id="mensajeLabel"`	`@FXML private Label mensajeLabel;`	El componente con el campo
`onAction="#onSaludarClick"`	`@FXML private void onSaludarClick()`	El evento con el método

¿En qué orden ocurre?

1. `FXMLLoader.load()` lee el archivo y crea los componentes.
2. Crea el controlador con su constructor sin parámetros.
3. Asigna cada campo `@FXML` cuyo nombre coincide con un `fx:id`.
4. Llama al método `initialize()` del controlador, **si existe**.
5. Cuando ocurre un evento (clic), llama al método indicado en `onAction`.

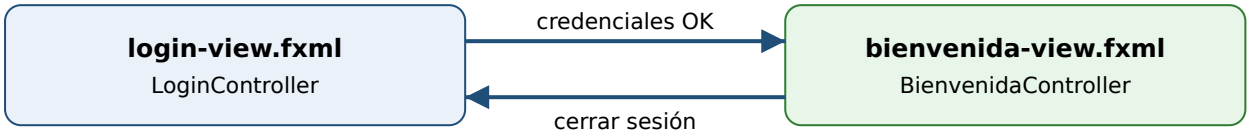
El método initialize()

```
@FXML
private void initialize() {
    // Se ejecuta UNA vez, cuando la vista ya está cargada y los @FXML asignados.
    // Aquí se prepara la pantalla: llenar un ComboBox, poner un texto inicial, etc.
    mensajeLabel.setText("Listo");
}
```

> En el **constructor** los campos `@FXML` todavía son `null`. Usa `initialize()` para trabajar con ellos.

7. Paso a paso 2: el Login

Objetivo: una pantalla de inicio de sesión. Si el usuario es `admin` y la clave `1234`, se muestra una pantalla de bienvenida; si no, aparece un mensaje de error.



Paso 1 · Diseñar la vista del login

Crea `login-view.fxml` en la carpeta de recursos y ábrelo en Scene Builder:

1. Raíz: **VBox** con Alignment = CENTER, Spacing = 12, Padding = 24.
2. Agrega un **Label** con el texto `Iniciar sesión` (título).
3. Agrega un **TextField**: *Prompt Text* = `Usuario` · *fx:id* = `usuarioField`.
4. Agrega un **PasswordField**: *Prompt Text* = `Clave` · *fx:id* = `claveField` · *On Action* = `onIngresarClick` (para poder ingresar con Enter).
5. Agrega un **Button**: *Text* = `Ingresar` · *On Action* = `onIngresarClick` · marca **Default Button**.
6. Agrega un **Label** vacío: *fx:id* = `errorLabel` (para los mensajes de error; se le puede poner color rojo en *Style*: `-fx-text-fill: red;`).
7. En *Controller class*: `com.duoc.javafxfxml.LoginController`.

login-view.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.control.PasswordField?>
<?import javafx.scene.control.TextField?>
<?import javafx.scene.layout.VBox?>

<VBox alignment="CENTER" spacing="12.0" xmlns="http://javafx.com/javafx/21"
xmlns:fx="http://javafx.com/fxml/1" fx:controller="com.duoc.javafxfxml.LoginController">
   <padding>
      <Insets bottom="24.0" left="24.0" right="24.0" top="24.0" />
   </padding>
   <children>
      <Label text="Iniciar sesión" style="-fx-font-size: 20px; -fx-font-weight: bold;" />
      <TextField fx:id="usuarioField" promptText="Usuario" />
      <PasswordField fx:id="claveField" promptText="Clave" onAction="#onIngresarClick" />
      <Button defaultButton="true" onAction="#onIngresarClick" text="Ingresar" />
      <Label fx:id="errorLabel" style="-fx-text-fill: red;" />
   </children>
</VBox>
```

Paso 2 · El controlador del login

LoginController.java

```
package com.duoc.javafxfxml;

import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.scene.control.PasswordField;
import javafx.scene.control.TextField;
import javafx.stage.Stage;

import java.io.IOException;

public class LoginController {

    // Usuario y clave fijos: solo para practicar (en un sistema real se validaría contra
una BD)
    private static final String USUARIO_VALIDO = "admin";
    private static final String CLAVE_VALIDA = "1234";

    @FXML
    private TextField usuarioField;

    @FXML
    private PasswordField claveField;

    @FXML
    private Label errorLabel;

    @FXML
    private void onIngresarClick() throws IOException {
        String usuario = usuarioField.getText().trim();
        String clave = claveField.getText();

        if (usuario.isEmpty() || clave.isEmpty()) {
            errorLabel.setText("Debes ingresar usuario y clave.");
            return;
        }

        if (USUARIO_VALIDO.equals(usuario) && CLAVE_VALIDA.equals(clave)) {
            abrirBienvenida(usuario);
        } else {
            errorLabel.setText("Usuario o clave incorrectos.");
            claveField.clear();
        }
    }

    private void abrirBienvenida(String usuario) throws IOException {
        FXMLLoader loader = new FXMLLoader(getClass().getResource("bienvenida-view.fxml"));
        Parent raiz = loader.load();

        // Podemos hablar con el otro controlador para pasarle datos
        BienvenidaController controlador = loader.getController();
        controlador.setUsuario(usuario);

        // La ventana actual se obtiene desde cualquier componente de la escena
```

```
        Stage stage = (Stage) usuarioField.getScene().getWindow();
        stage.setTitle("Bienvenida");
        stage.setScene(new Scene(raiz, 360, 280));
    }
}
```

Lo importante de este controlador

- **Validación:** primero se comprueba que los campos no estén vacíos y luego se comparan con los valores válidos. Se escribe `USUARIO_VALIDO.equals(usuario)` (y no al revés) para evitar errores si `usuario` fuera `null`.
- **Cambio de pantalla:** se carga otro FXML con otro `FXMLLoader` y se reemplaza la escena de la *misma* ventana con `stage.setScene(...)`.
- **Obtener la ventana:** `usuarioField.getScene().getWindow()`. Cualquier componente conoce su escena, y la escena conoce su ventana.
- **Pasar datos al otro controlador:** `loader.getController()` devuelve el controlador de la nueva vista, y se le llama a un método (`setUsuario`).

> En un sistema real **nunca** se dejan usuario y clave escritos en el código. Aquí es solo para practicar: se validaría contra una base de datos y la clave estaría cifrada.

Paso 3 · La vista y el controlador de bienvenida

bienvenida-view.fxml

```
<?xml version="1.0" encoding="UTF-8"?>

<?import javafx.geometry.Insets?>
<?import javafx.scene.control.Button?>
<?import javafx.scene.control.Label?>
<?import javafx.scene.layout.VBox?>

<VBox alignment="CENTER" spacing="20.0" xmlns="http://javafx.com/javafx/21"
xmlns:fx="http://javafx.com/fxml/1"
fx:controller="com.duoc.javafxfxml.BienvenidaController">
   <padding>
      <Insets bottom="24.0" left="24.0" right="24.0" top="24.0" />
   </padding>
   <children>
      <Label fx:id="saludoLabel" style="-fx-font-size: 18px;" text="¡Bienvenido!" />
      <Button onAction="#onCerrarSesionClick" text="Cerrar sesión" />
   </children>
</VBox>
```

BienvenidaController.java

```
package com.duoc.javafxfxml;

import javafx.fxml.FXML;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.scene.control.Label;
import javafx.stage.Stage;
```

```
import java.io.IOException;

public class BienvenidaController {

    @FXML
    private Label saludoLabel;

    public void setUsuario(String usuario) {
        saludoLabel.setText("¡Bienvenido, " + usuario + "!");
    }

    @FXML
    private void onCerrarSesionClick() throws IOException {
        FXMLLoader loader = new FXMLLoader(getClass().getResource("login-view.fxml"));
        Parent raiz = loader.load();

        Stage stage = (Stage) saludoLabel.getScene().getWindow();
        stage.setTitle("Login");
        stage.setScene(new Scene(raiz, 360, 280));
    }
}
```

Paso 4 · La clase Application del login

LoginApplication.java

```
package com.duoc.javafxfxml;

import javafx.application.Application;
import javafx.fxml.FXMLLoader;
import javafx.scene.Parent;
import javafx.scene.Scene;
import javafx.stage.Stage;

import java.io.IOException;

/** Aplicación de login: la ventana parte mostrando login-view.fxml. */
public class LoginApplication extends Application {

    @Override
    public void start(Stage stage) throws IOException {
        FXMLLoader loader = new FXMLLoader(getClass().getResource("login-view.fxml"));
        Parent raiz = loader.load();

        stage.setTitle("Login");
        stage.setScene(new Scene(raiz, 360, 280));
        stage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }
}
```

Paso 5 · Probar

Prueba	Resultado esperado
Campos vacíos → Ingresar	“Debes ingresar usuario y clave.”
Usuario `admin` y clave incorrecta	“Usuario o clave incorrectos.” y la clave se borra.
`admin` / `1234`	Aparece “¡Bienvenido, admin!”.
Cerrar sesión	Vuelve al login.

8. Elementos y métodos importantes

8.1 Anotaciones y clases de FXML

Elemento	¿Qué es / para qué sirve?
`@FXML`	Anotación. Marca campos y métodos del controlador que la vista puede usar, aunque sean `private`.
`FXMLLoader`	Clase que lee un `.fxml` y construye la interfaz.
`fx:id`	Atributo del FXML: nombre del componente para el controlador.
`fx:controller`	Atributo del FXML: clase controladora (en el nodo raíz).
`onAction="#metodo"`	Atributo del FXML: método que responde al evento principal.
`Parent`	Tipo de dato que devuelve `load()`: el nodo raíz de la vista.

8.2 Métodos importantes

Método	¿Qué hace?
`new FXMLLoader(URL)`	Crea el cargador con la ubicación del archivo.
`getClass().getResource("x.fxml")`	Obtiene la ruta del archivo dentro de los recursos del proyecto. Si el nombre está mal devuelve `null`.
`loader.load()`	Construye la vista. Puede lanzar `IOException`.
`loader.getController()`	Devuelve el controlador asociado (para pasarle datos).
`initialize()`	Método (opcional) del controlador que se ejecuta al terminar de cargar.
`stage.setScene(escena)`	Cambia lo que muestra la ventana (navegar entre pantallas).
`nodo.getScene().getWindow()`	Obtiene la ventana desde cualquier componente.
`campo.clear()`	Limpia un campo de texto.

9. Ejecutar el proyecto

9.1 Por línea de comandos

1. Abre una terminal y entra a la carpeta del proyecto:

   ```
   cd 2_javafx-fxml
   ```

2. Ejecuta el login (clase principal por defecto del `pom.xml`):

   ```
   mvn javafx:run
   ```

3. Para ejecutar la demo mínima:

   ```
   mvn javafx:run -Dmain.class=com.duoc.javafxfxml.HolaApplication
   ```

4. Para solo compilar: `mvn clean compile`

Windows PowerShell: si el parámetro `-D` da problemas, escríbelo entre comillas: `mvn javafx:run "-Dmain.class=com.duoc.javafxfxml.HolaApplication"`.

9.2 Con IntelliJ IDEA

1. *File* → *Open...* → selecciona la carpeta (o el `pom.xml`) → *Open as Project*.
2. Espera a que Maven termine de importar.
3. Verifica el JDK 21 en *File* → *Project Structure* → *Project*.
4. Abre `LoginApplication.java` o `HolaApplication.java` y presiona el triángulo verde junto a `main`.
5. Alternativa: panel **Maven** → *Plugins* → *javafx* → *javafx:run*.

> Si cambias el FXML y no ves el cambio: *Build* → *Rebuild Project* o `mvn clean compile`. Los FXML se copian a `target/classes` al compilar.

10. Errores frecuentes

Mensaje / síntoma	Causa	Solución
`Location is not set` / `NullPointerException` al crear `FXMLLoader`	El nombre o la carpeta del `.fxml` no coincide.	Revisar que esté en `resources/com/duoc/javafxfxml` y el nombre exacto. Ejecutar `mvn clean compile`.
`Module ... does not open ... to module javafx.fxml`	Falta el `opens` en `module-info.java`.	Agregar `opens com.duoc.javafxfxml to javafx.fxml;`.
`Module javafx.fxml not found`	Falta la dependencia o el `requires`.	Agregar `javafx-fxml` al `pom.xml` y `requires javafx.fxml;`. Recargar Maven.
`NullPointerException` al usar un campo `@FXML`	El `fx:id` no coincide con el nombre del campo, o se usa en el constructor.	Igualar los nombres (distingue mayúsculas) y usar `initialize()`.
`No controller specified`	Falta `fx:controller` en la raíz.	Indicarlo en Scene Builder (*Controller class*).
`LoadException ... Error resolving onAction='#...'`	El método indicado no existe en el controlador.	Crear el método con `@FXML` y el mismo nombre.
`ClassNotFoundException` del controlador	El nombre completo del controlador está mal escrito.	Revisar paquete y nombre de la clase.
Scene Builder no abre desde IntelliJ	Ruta mal configurada.	Revisar *Settings → Languages & Frameworks → JavaFX*.

11. Resumen y checklist

Para crear una pantalla nueva con FXML

1. Crear `nombre-view.fxml` en `src/main/resources/com/duoc/javafxfxml`.
2. Diseñarla en Scene Builder; asignar **fx:id**, **On Action** y **Controller class**.
3. Crear la clase `NombreController` en el mismo paquete que las demás.
4. Declarar los campos `@FXML` con los mismos nombres de los `fx:id`.
5. Crear los métodos `@FXML` con los mismos nombres de los eventos.
6. Cargar con `FXMLLoader` y mostrar con `Scene` y `Stage`.
7. Ejecutar con `mvn javafx:run`.

> **Idea para recordar:** la **vista** (FXML) dice *qué hay*; el **controlador** dice *qué pasa*; el **FXMLLoader** los une por nombres.